Database Management System

Practical Copy

PL/SQL & SQL Programs

Date: 13/04/26

Q1. Find the greater of two integers using IF-THEN-ELSE

Aim: Write a PL/SQL block to find the greater number between two integers using an IF-THEN-ELSE statement.

```
DECLARE
   a NUMBER := 25;
   b NUMBER := 40;
BEGIN
   IF a > b THEN
      DBMS_OUTPUT.PUT_LINE('Greater number is: ' || a);
   ELSE
      DBMS_OUTPUT.PUT_LINE('Greater number is: ' || b);
   END IF;
END;
/
```

Output:

```
Greater number is: 40
```

Q2. Greatest of three numbers using nested IF

Aim: Write a PL/SQL block to find the greatest of three numbers using nested IF statements.

```
DECLARE
   a NUMBER := 15;
   b NUMBER := 38;
   c NUMBER := 27;
BEGIN
   IF a >= b AND a >= c THEN
      DBMS OUTPUT.PUT LINE('Greatest number is: ' || a);
   ELSE
      IF b >= a AND b >= c THEN
         DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || b);
      ELSE
         DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || c);
      END IF;
   END IF;
END;
/
```

Output:

```
Greatest number is: 38
```

Q3. Greatest of three numbers using ELSIF and logical AND

Aim: Write a PL/SQL block to find the greatest of three numbers using ELSIF and logical AND operators.

```
DECLARE
   a NUMBER := 50;
```

```
   b NUMBER := 73;
   c NUMBER := 61;
BEGIN
   IF a >= b AND a >= c THEN
      DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || a);
   ELSIF b >= a AND b >= c THEN
      DBMS_OUTPUT.PUT_LINE('Greatest number is: ' || b);
   ELSE
      DBMS OUTPUT.PUT LINE('Greatest number is: ' || c);
   END IF;
END;
/
```

Output:

```
Greatest number is: 73
```

Q4. Sum of first 10 natural numbers using basic LOOP

Aim: Write a PL/SQL block to find the sum of the first ten natural numbers (1 to 10) using an unconditional basic LOOP.

```
DECLARE
   i   NUMBER := 1;
   sum NUMBER := 0;
BEGIN
   LOOP
      sum := sum + i;
      i   := i + 1;
      EXIT WHEN i > 10;
   END LOOP;
   DBMS OUTPUT.PUT LINE('Sum of first 10 natural numbers = ' || sum);
END;
/
```

Output:

```
Sum of first 10 natural numbers = 55
```

Q5. Sum of first 10 natural numbers using WHILE loop

Aim: Write a PL/SQL block to find the sum of the first ten natural numbers (1 to 10) using a conditional WHILE loop.

```
DECLARE
   i   NUMBER := 1;
   sum NUMBER := 0;
BEGIN
   WHILE i <= 10 LOOP
      sum := sum + i;
      i   := i + 1;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Sum of first 10 natural numbers = ' || sum);
END;
/
```

Output:

```
Sum of first 10 natural numbers = 55
```

Q6. Sum of first 10 natural numbers using numeric FOR loop

Aim: Write a PL/SQL block to find the sum of the first ten natural numbers (1 to 10) using a numeric FOR loop.

```
DECLARE
   sum NUMBER := 0;
BEGIN
   FOR i IN 1..10 LOOP
      sum := sum + i;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Sum of first 10 natural numbers = ' || sum);
END;
/
```

Output:

```
Sum of first 10 natural numbers = 55
```

Q7. Print numbers 10 down to 1 using REVERSE FOR loop

Aim: Write a PL/SQL block to print numbers from 10 down to 1 using a REVERSE FOR loop.

```
BEGIN
   FOR i IN REVERSE 1..10 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
   END LOOP;
END;
/
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

Q8. Retrieve first_name and salary from HR_Employees using cursor FOR loop

Aim: Write a PL/SQL block to retrieve and display the first_name and salary of all records from the HR_Employees table using a cursor FOR loop.

```
DECLARE
   CURSOR emp_cursor IS
      SELECT first_name, salary
      FROM   HR_Employees;
BEGIN
   FOR emp_rec IN emp_cursor LOOP
      DBMS_OUTPUT.PUT_LINE('Name: ' || emp_rec.first_name ||
                           '  |  Salary: ' || emp rec.salary);
   END LOOP;
END;
/
```

Output:

```
Name: Steven     |  Salary: 24000
Name: Neena      |  Salary: 17000
Name: Lex        |  Salary: 17000
Name: Alexander  |  Salary: 9000
... (all employee records displayed)
```

Date: 15/04/26

Q9. Factorial of a given number

Aim: Write a PL/SQL block to find the factorial of a given number.

```
DECLARE
   n       NUMBER := 6;
   fact    NUMBER := 1;
   i       NUMBER;
BEGIN
   IF n < 0 THEN
      DBMS_OUTPUT.PUT_LINE('Factorial is not defined for negative numbers.');
   ELSE
      FOR i IN 1..n LOOP
         fact := fact * i;
      END LOOP;
      DBMS_OUTPUT.PUT_LINE('Factorial of ' || n || ' = ' || fact);
   END IF;
END;
/
```

Output:

```
Factorial of 6 = 720
```

Q10. Check whether a number is prime or not

Aim: Write a PL/SQL block to check whether a given number is prime or not.

```
DECLARE
   n       NUMBER := 17;
   i       NUMBER;
   flag    NUMBER := 0;
BEGIN
   IF n <= 1 THEN
      flag := 1;
   ELSE
      FOR i IN 2..FLOOR(SQRT(n)) LOOP
         IF MOD(n, i) = 0 THEN
            flag := 1;
            EXIT;
         END IF;
      END LOOP;
   END IF;
   IF flag = 0 THEN
      DBMS_OUTPUT.PUT_LINE(n || ' is a Prime number.');
   ELSE
      DBMS_OUTPUT.PUT_LINE(n || ' is NOT a Prime number.');
   END IF;
END;
/
```

Output:

```
17 is a Prime number.
```

Q11. Check whether a number is an Armstrong number

Aim: Write a PL/SQL block to check whether an inputted number is an Armstrong number or not.

```
DECLARE
   n        NUMBER := 153;
```

```
   temp     NUMBER;
   digit    NUMBER;
   digits   NUMBER := 0;
   sum_pow  NUMBER := 0;
BEGIN
   temp := n;
   -- Count digits
   WHILE temp > 0 LOOP
      digits := digits + 1;
      temp   := FLOOR(temp / 10);
   END LOOP;
   temp := n;
   -- Sum of digits raised to power
   WHILE temp > 0 LOOP
      digit   := MOD(temp, 10);
      sum_pow := sum_pow + POWER(digit, digits);
      temp    := FLOOR(temp / 10);
   END LOOP;
   IF sum_pow = n THEN
      DBMS OUTPUT.PUT LINE(n || ' is an Armstrong number.');
   ELSE
      DBMS_OUTPUT.PUT_LINE(n || ' is NOT an Armstrong number.');
   END IF;
END;
/
```

Output:

```
153 is an Armstrong number.
```

Q12. Reverse a given integer number

Aim: Write a PL/SQL block to reverse a given integer number (e.g., 123 becomes 321).

```
DECLARE
   n        NUMBER := 12345;
   rev      NUMBER := 0;
   digit    NUMBER;
   temp     NUMBER;
BEGIN
   temp := n;
   WHILE temp > 0 LOOP
      digit := MOD(temp, 10);
      rev   := rev * 10 + digit;
      temp  := FLOOR(temp / 10);
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Original number : ' || n);
   DBMS_OUTPUT.PUT_LINE('Reversed number : ' || rev);
END;
/
```

Output:

```
Original number : 12345
Reversed number : 54321
```

Q13. Reverse a character string variable

Aim: Write a PL/SQL block to reverse a given character string variable.

```
DECLARE
   str     VARCHAR2(100) := 'ORACLE';
   rev str VARCHAR2(100) := '';
   i       NUMBER;
BEGIN
```

```
   FOR i IN REVERSE 1..LENGTH(str) LOOP
      rev_str := rev_str || SUBSTR(str, i, 1);
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Original string : ' || str);
   DBMS_OUTPUT.PUT_LINE('Reversed string : ' || rev_str);
END;
/
```

Output:

```
Original string : ORACLE
Reversed string : ELCARO
```

Q14. Generate and display the multiplication table

Aim: Write a PL/SQL block to generate and display the multiplication table of a given number.

```
DECLARE
   n NUMBER := 7;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Multiplication Table of ' || n);
   DBMS_OUTPUT.PUT_LINE('------------------------------');
   FOR i IN 1..10 LOOP
      DBMS_OUTPUT.PUT_LINE(n || ' x ' || i || ' = ' || (n * i));
   END LOOP;
END;
/
```

Output:

```
Multiplication Table of 7
------------------------------
7 x 1  = 7
7 x 2  = 14
7 x 3  = 21
7 x 4  = 28
7 x 5  = 35
7 x 6  = 42
7 x 7  = 49
7 x 8  = 56
7 x 9  = 63
7 x 10 = 70
```

Q15. Division with ZERO_DIVIDE exception handling

Aim: Write a PL/SQL block to accept two numbers and display their division. Handle the ZERO_DIVIDE exception to print a custom error message if the divisor is zero.

```
DECLARE
   a      NUMBER := 100;
   b      NUMBER := 0;
   result NUMBER;
BEGIN
   result := a / b;
   DBMS_OUTPUT.PUT_LINE('Result = ' || result);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed!');
      DBMS_OUTPUT.PUT_LINE('Please provide a non-zero divisor.');
END;
/
```

Output:

```
Error: Division by zero is not allowed!
Please provide a non-zero divisor.
```

Q16. Voter age validation using user-defined exception

Aim: Write a PL/SQL block for voter age validation. Declare a user-defined exception named INVALID_AGE, raise it explicitly if the inputted age is less than 18, and handle it by printing a "Cannot vote" message.

```
DECLARE
   age         NUMBER := 15;
   INVALID_AGE EXCEPTION;
BEGIN
   IF age < 18 THEN
      RAISE INVALID_AGE;
   ELSE
      DBMS_OUTPUT.PUT_LINE('Age: ' || age);
      DBMS_OUTPUT.PUT_LINE('You are eligible to vote.');
   END IF;
EXCEPTION
   WHEN INVALID_AGE THEN
      DBMS_OUTPUT.PUT_LINE('Age: ' || age);
      DBMS OUTPUT.PUT LINE('Cannot vote. You must be at least 18 years old.');
END;
/
```

Output:

```
Age: 15
Cannot vote. You must be at least 18 years old.
```

Date: 20/04/26

SQL Queries — Built-in Functions

Q17. Miscellaneous & Value Comparison Functions (DUAL table)

Aim: Write SQL queries using the DUAL table to find: lowest value, highest value, lowest alphabetical value, and earlier date.

```
-- (i) Lowest value from the list: 10, 10.5, -10, -100, 0
SELECT LEAST(10, 10.5, -10, -100, 0) AS Lowest_Value FROM DUAL;
```

Output:

```
LOWEST VALUE
------------
        -100
```

```
-- (ii) Highest value from the same list
SELECT GREATEST(10, 10.5, -10, -100, 0) AS Highest_Value FROM DUAL;
```

Output:

```
HIGHEST_VALUE
-------------
         10.5
```

```
-- (iii) Lowest alphabetical value among 'a', 'b', 'c', 'B'
SELECT LEAST('a', 'b', 'c', 'B') AS Lowest_Alpha FROM DUAL;
```

Output:

```
LOWEST_ALPHA
------------
B
```

```
-- (iv) Earlier date between '19-Apr-26' and SYSDATE
SELECT LEAST(TO_DATE('19-Apr-26','DD-Mon-RR'), SYSDATE) AS Earlier_Date FROM DUAL;
```

Output:

```
EARLIER_DATE
------------
19-APR-26
```

Q18. String Manipulation Functions (DUAL table)

Aim: Write SQL queries using the DUAL table to perform: lowercase/uppercase conversion, initial capital letters, and character length.

```
-- (i) Convert 'SIGN' to lowercase and 'sign' to uppercase
SELECT LOWER('SIGN') AS lowercase_sign,
       UPPER('sign') AS uppercase_sign
FROM DUAL;
```

Output:

```
LOWERCASE_SIGN   UPPERCASE_SIGN
--------------   --------------
sign             SIGN
```

```
-- (ii) Convert 'patna science college' to initial capital letters
SELECT INITCAP('patna science college') AS Initcap_String FROM DUAL;
```

Output:

```
INITCAP_STRING
----------------------
Patna Science College
```

```
-- (iii) Find the character length of the string 'science'
SELECT LENGTH('science') AS String_Length FROM DUAL;
```

Output:

```
STRING_LENGTH
-------------
            7
```

Q19. Character & ASCII Conversions (DUAL table)

Aim: Write SQL queries using the DUAL table to: display character for ASCII 65, find ASCII value of 'A', and explain ASCII of 'abc'.

```
-- (i) Display the character corresponding to ASCII code 65
SELECT CHR(65) AS Character_65 FROM DUAL;
```

Output:

```
CHARACTER_65
------------
A
```

```
-- (ii) Find the ASCII value of the character 'A'
SELECT ASCII('A') AS ASCII_Value FROM DUAL;
```

Output:

```
ASCII_VALUE
-----------
         65
```

```
-- (iii) Find the ASCII value of the string 'abc' and explain which character's
value is returned
SELECT ASCII('abc') AS ASCII_of_abc FROM DUAL;
```

Output:

```
ASCII_OF_ABC
------------
          97
```

Note: *ASCII() function in Oracle returns the decimal representation of the FIRST character of the input string. Here 'abc' starts with 'a', so ASCII('abc') = 97 (ASCII value of 'a').*

Q20. Substring & Instrument Functions (SUBSTR & INSTR)

Aim: Write SQL queries using the DUAL table to demonstrate pattern searching: extract substrings and find positions of characters.

```
-- (i) Extract substring from 'college' starting at 4th position, length 3
SELECT SUBSTR('college', 4, 3) AS With_Length,
       SUBSTR('college', 4)    AS Without_Length
FROM DUAL;
```

Output:

```
WITH_LENGTH   WITHOUT_LENGTH
-----------   --------------
leg           lege
```

```
-- (ii) Find position of 1st occurrence of 'e' in 'college'
SELECT INSTR('college', 'e') AS First_Occurrence FROM DUAL;
```

Output:

```
FIRST_OCCURRENCE
----------------
               6
```

```
-- (iii) Find position of 2nd occurrence of 'e' in 'college',
--       scanning forward from position 1 and 6th position
SELECT INSTR('college', 'e', 1, 2) AS Second_Occurrence FROM DUAL;
```

Output:

```
SECOND_OCCURRENCE
-----------------
                8
```

Q21. Phonetic Matching using SOUNDEX

Aim: Write a SQL query to retrieve the first_name and last_name from hr_employees where the first name sounds phonetically identical to 'Alan' or 'Alen'.

```
SELECT first_name, last_name
FROM   hr_employees
WHERE  SOUNDEX(first_name) = SOUNDEX('Alan')
   OR  SOUNDEX(first_name) = SOUNDEX('Alen');
```

Output:

```
FIRST_NAME   LAST_NAME
----------   ---------
Allan        McEwen
(rows with phonetically similar names to 'Alan' or 'Alen')
```

Note: *SOUNDEX() converts a string to a phonetic code. Both 'Alan' and 'Alen' produce the same SOUNDEX code (A450), so a single condition SOUNDEX(first_name) = SOUNDEX('Alan') is sufficient to match all phonetically identical names.*

--- End of Practical Copy ---